Министерство образования Республики Беларусь

Учреждение образования
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»

Факультет информационных технологий и управления
Кафедра информационных технологий автоматизированных систем

По контрольной работе №1

Выполнил: ст. гр. 320604
М.И. Якубовский

Проверила: Н. В. Хаджинова

Минск 2026

1 ЦЕЛЬ РАБОТЫ

Разработать REST API на Node.js с аутентификацией (JWT), работой с реляционной базой данных (PostgreSQL) и реализацией CRUD-операций для сущности, соответствующей теме курсового проекта (система управления учебными заданиями и успеваемостью студентов).

2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

В рамках серверной разработки на Node.js с использованием фреймворка Express.js создаются надёжные и масштабируемые REST API. Для хранения данных применяется облачная реляционная база данных PostgreSQL (Neon.tech), взаимодействие с которой осуществляется через ORM-библиотеку Sequelize. Для обеспечения безопасности реализуется аутентификация на основе JWT (JSON Web Token), защита маршрутов с помощью промежуточных обработчиков (middleware) и разграничение прав доступа на основе ролей (Role-Based Access Control, RBAC). Дополнительно применяются механизмы валидации входящих параметров с использованием библиотеки Joi, а также инструменты для фильтрации и контекстного поиска выборок из базы данных.

3 ХОД РАБОТЫ

Выполнение работы началось с проектирования реляционной базы данных. Были созданы следующие основные сущности:

Users — для хранения учётных данных пользователей, включая уникальное поле email, защищённое поле с хешем пароля passwordHash, роль role (user, moderator, admin), а также поля защиты от перебора паролей failedAttempts и lockUntil.

Assignments — для хранения учебных заданий с атрибутами title (название), description (подробное описание), status (статус выполнения: «Выдано» или «Сдано») и grade (оценка).

Таблицы связаны между собой связями «один-ко-многим»: пользователь-студент может иметь множество назначенных заданий через внешний ключ studentId, а преподаватель — множество выданных заданий через внешний ключ teacherId).

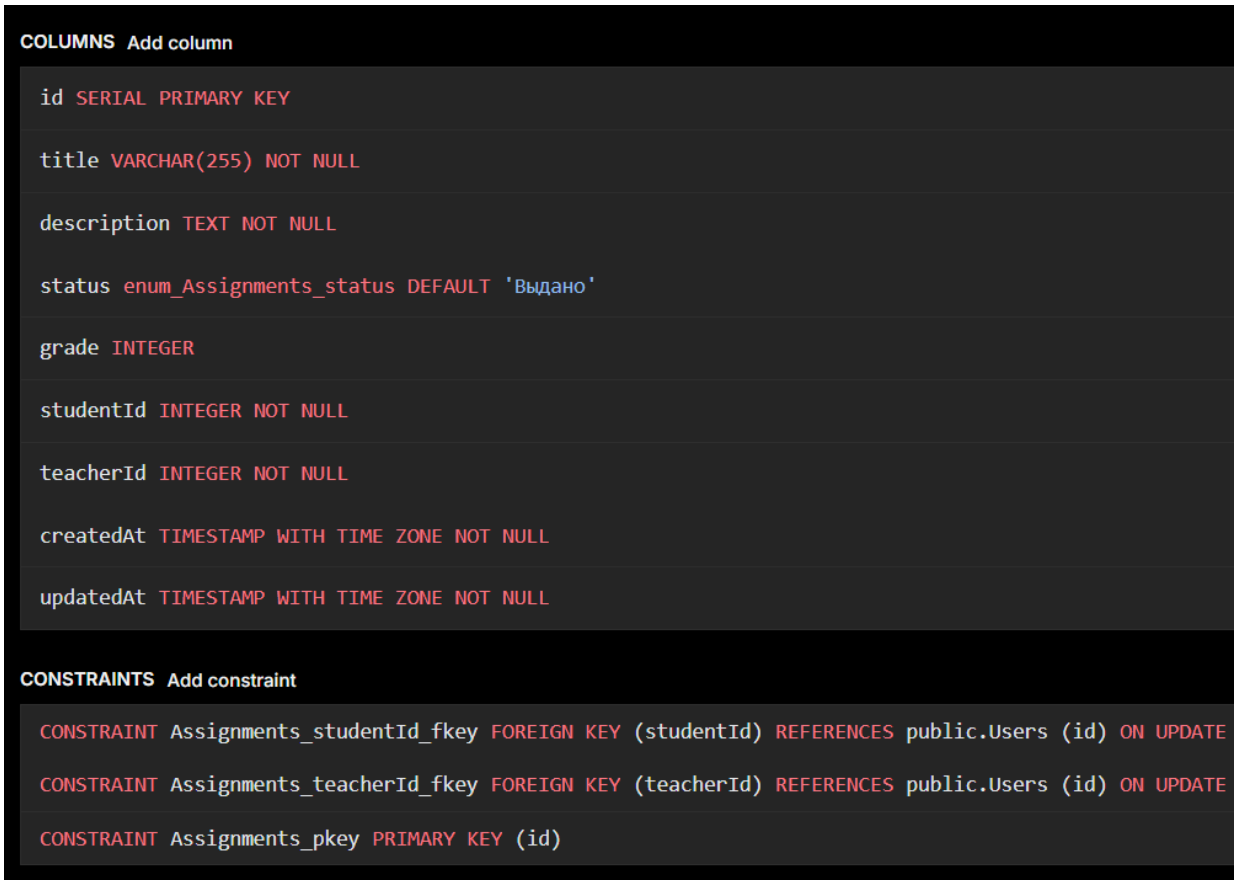


Рисунок 1 – Архитектура базы данных

Далее была реализована бизнес-логика приложения. В файле models/Assignment.js описана структура сущности задания. В контроллере controllers/assignmentController.js реализованы операции создания, чтения, детального просмотра по идентификатору, обновления и удаления заданий, а также механизмы фильтрации по статусу и поиска по названию. Разграничение прав доступа настроено в middlewares/authMiddleware.js, а валидация входящих схем данных — через middlewares/validate.js с помощью библиотеки Joi.

Авторизация и получение токена: был отправлен POST-запрос на адрес http://localhost:3000/auth/login с передачей учетных данных в теле запроса. Сервер успешно вернул статус 200 OK и сгенерированный JWT-токен (рисунок 2).

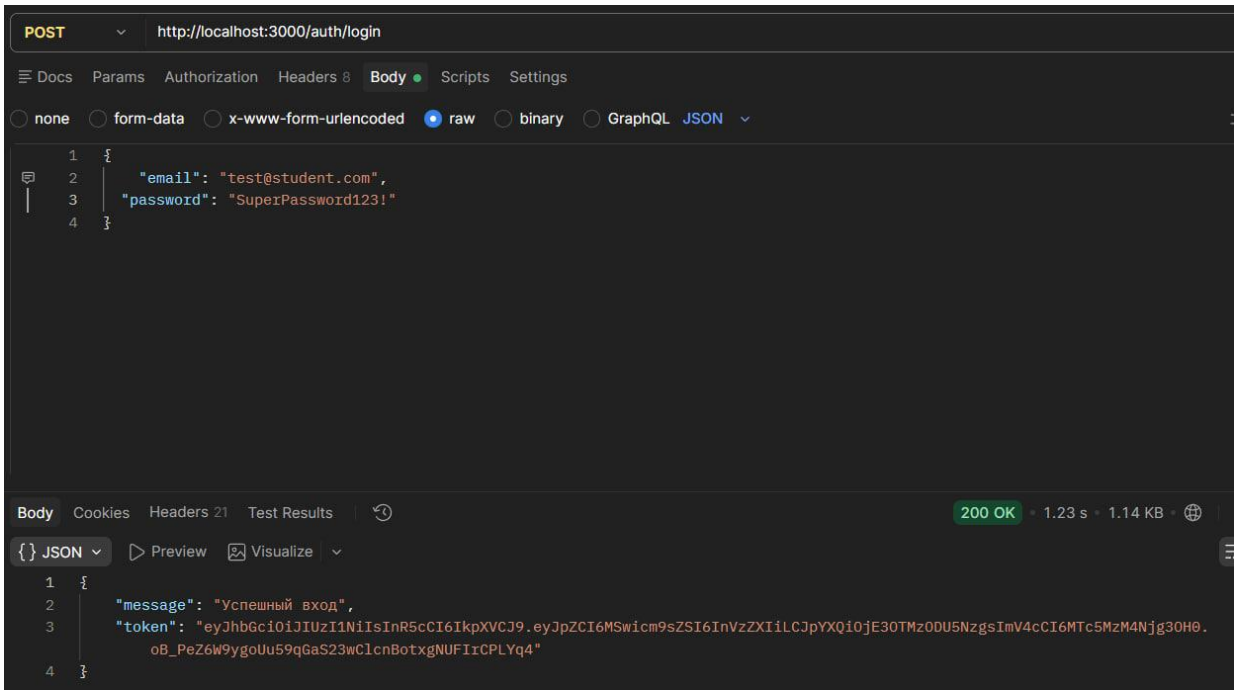


Рисунок 2 – Авторизация и получение токена

Защита маршрутов: при попытке выполнить GET-запрос к защищённому ресурсу http://localhost:3000/assignments без указания заголовка Authorization сервер отклонил запрос со статусом 401 Unauthorized (рисунок 3).

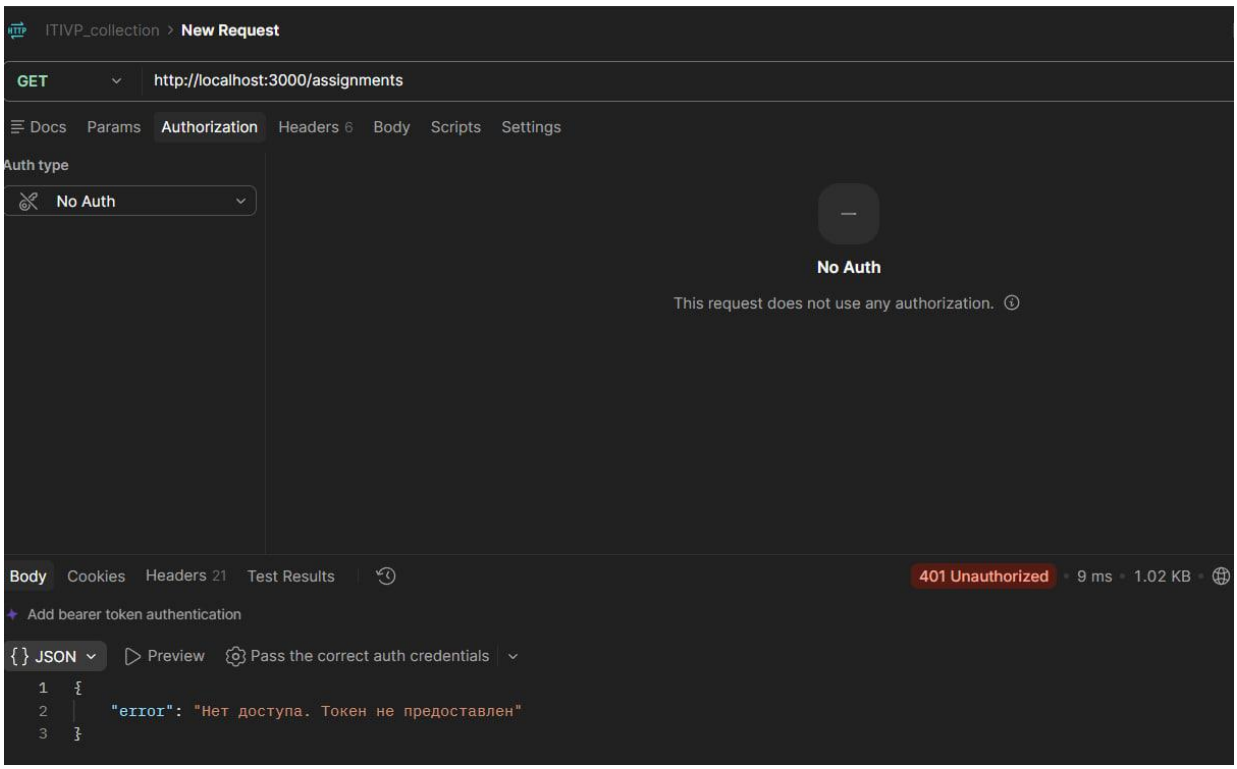


Рисунок 3 – Доступ к защищенному ресурсу без токена

Создание задания: авторизованный преподаватель выполнил POST-запрос на http://localhost:3000/assignments, передав название, описание и ID студента. Сервер сохранил запись в БД и вернул созданный объект со статусом 201 Created (рисунок 4).

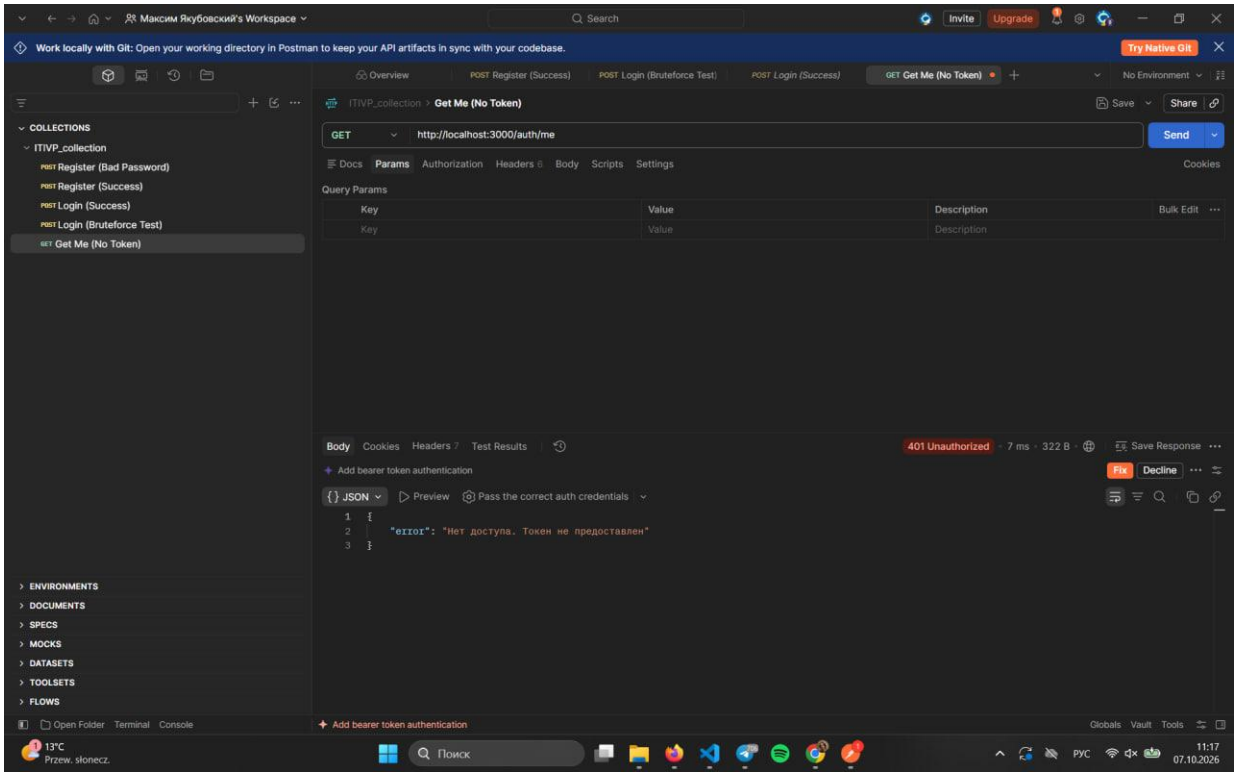


Рисунок 4 – Создание записи

Фильтрация и поиск (дополнительный эндпоинт): был выполнен GET-запрос с query-параметрами http://localhost:3000/assignments?status=Выдано&search=Лаб.
Сервер успешно отфильтровал выборку по статусу и совпадению подстроки в названии, вернув статус 200 OK. (рисунок 5).

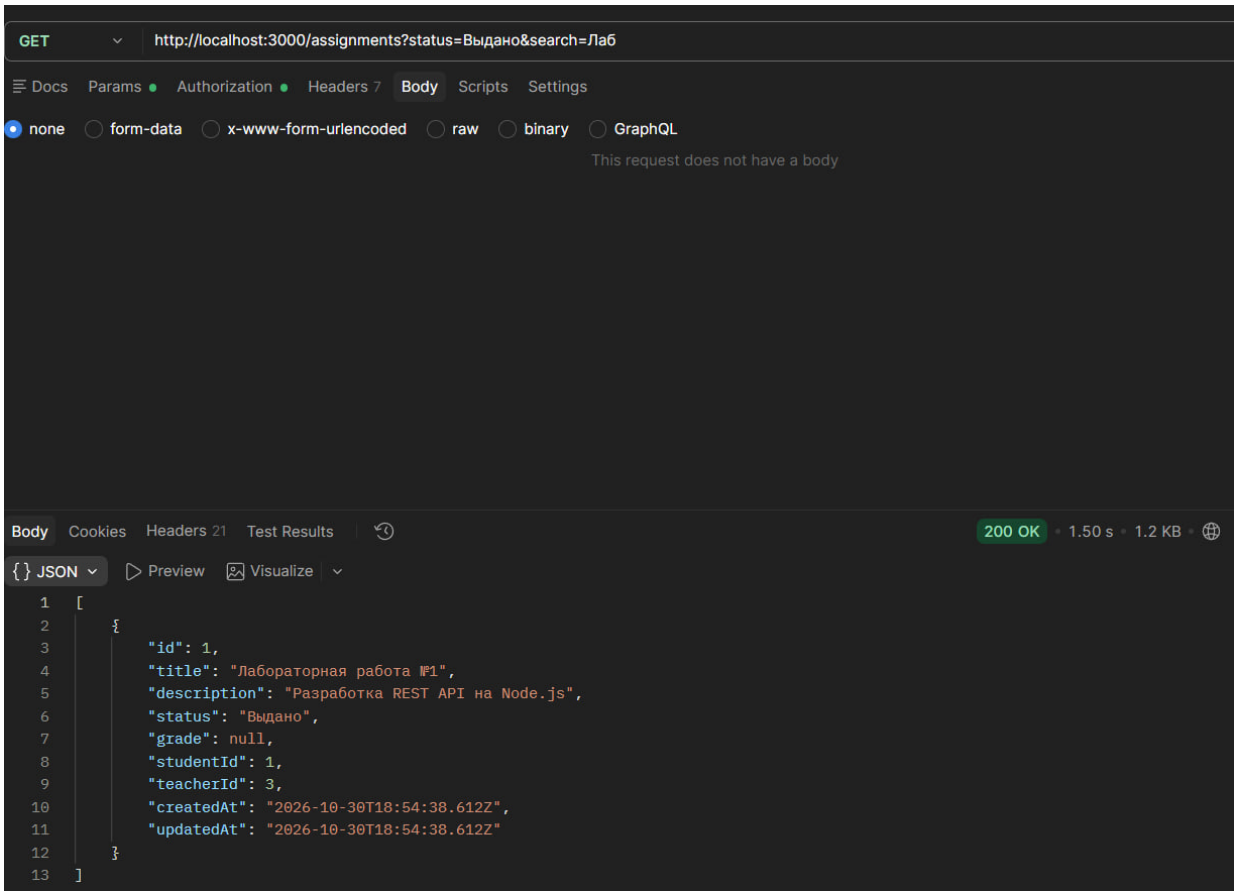


Рисунок 5 – Фильтрация и поиск записей

Ошибка валидации данных: при попытке отправить POST-запрос с некорректным телом (название короче 3 символов или отрицательная оценка) middleware валидации Joi перехватил запрос и вернул клиенту статус 400 Bad Request с пояснением ошибки (рисунок 6).

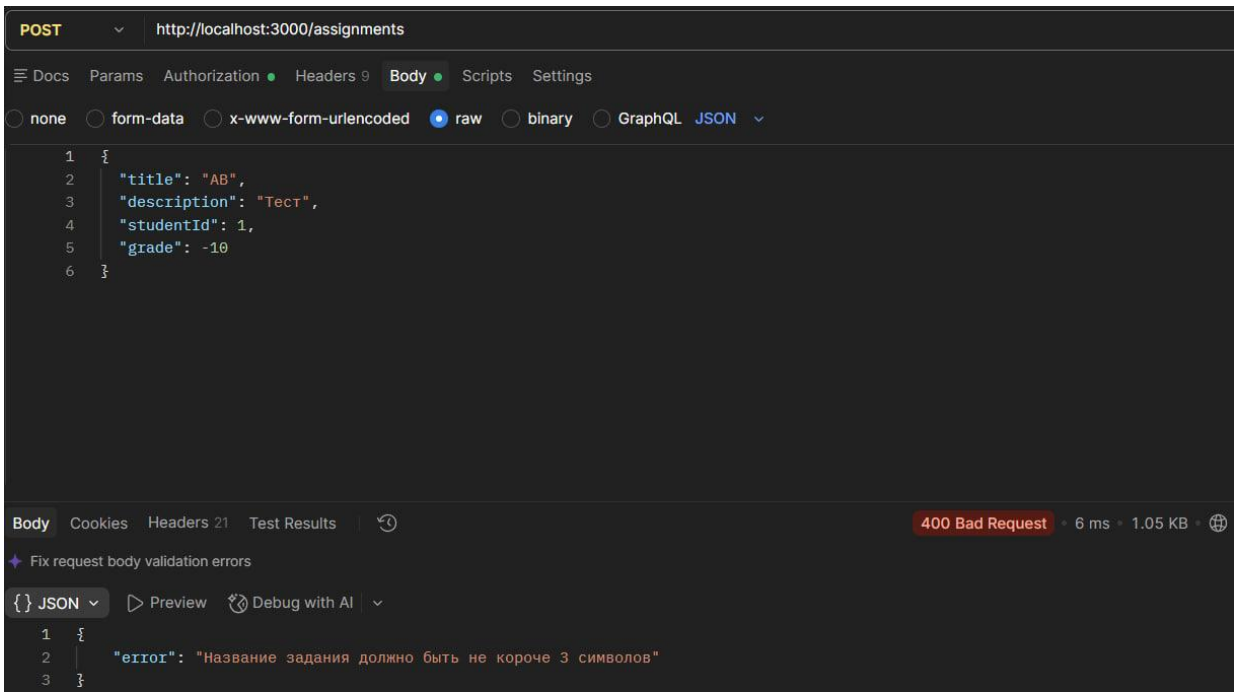


Рисунок 6 – Ошибка валидации входных данных

Удаление записи и контроль ролей: при попытке удаления задания пользователем без роли администратора система заблокировала операцию со статусом 403 Forbidden, подтвердив работу RBAC (рисунок 7).

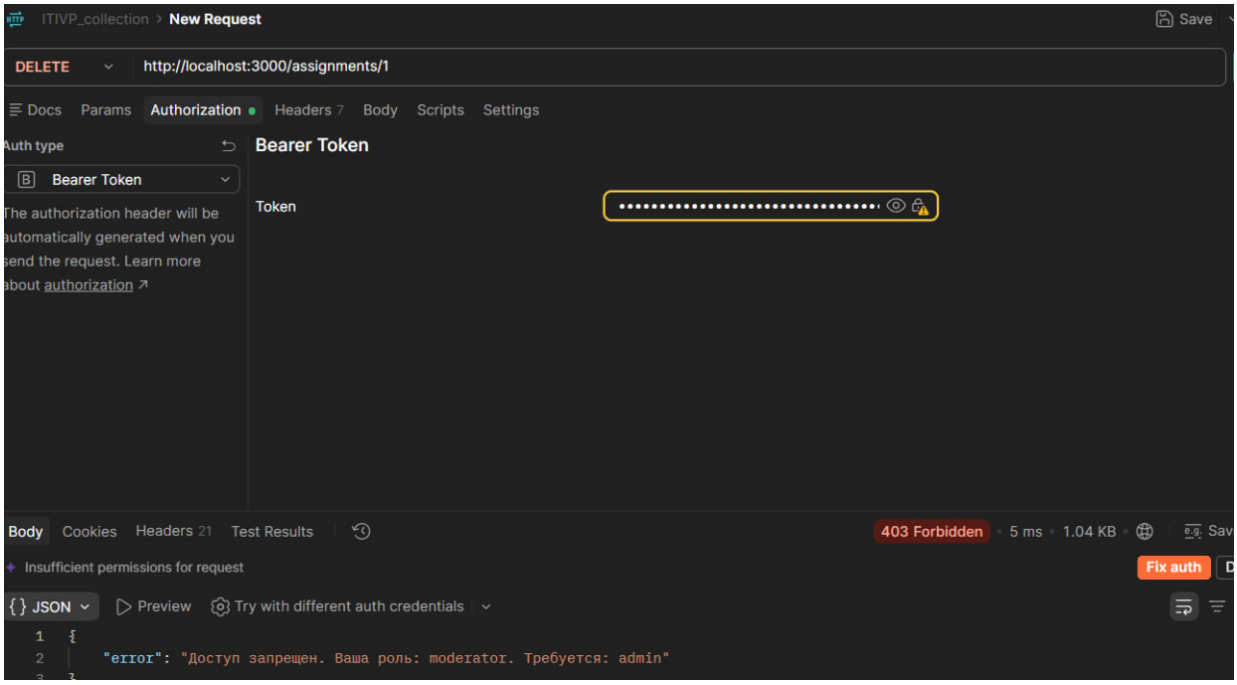


Рисунок 7 – Проверка прав доступа при удалении записи

ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

3.1 Какую сущность вы выбрали для реализации CRUD и как она связана с темой вашего курсового проекта?

Была выбрана сущность Assignment (Учебное задание), являющаяся ключевым элементом разрабатываемой системы управления заданиями и успеваемостью. Сущность связана связью «один-ко-многим» с пользователями (Users): со студентом через поле studentId и с преподавателем через поле teacherId.

3.2 Какие технологии вы использовали для разработки API и почему?

Использована серверная платформа Node.js и легковесный фреймворк Express.js, обеспечивающие высокую скорость разработки и удобную работу с middleware. В качестве СУБД задействована облачная реляционная база PostgreSQL (Neon.tech) в связке с ORM Sequelize, что позволило автоматизировать миграцию схемы, валидацию типов и построение безопасных SQL-запросов. Валидация входных данных выполнена через библиотеку Joi.

3.3 Как организована аутентификация? Опишите процесс регистрации и входа.

Аутентификация организована на базе протокола JWT (JSON Web Token). При регистрации пароль хешируется с добавлением соли через bcrypt и сохраняется в БД. При входе введённый пароль сопоставляется с хешем; при успешной проверке генерируется подписанный токен с ограниченным сроком жизни, содержащий идентификатор и роль пользователя. Для доступа к защищённым ресурсам клиент обязан передавать данный токен в заголовке Authorization: Bearer <TOKEN>.

Репозиторий проекта на *GitHub*:
https://github.com/Fira-git/University_7_itivp_dot/tree/kr21
Коллекция запросов *Postman*:
https://github.com/Fira-git/University_7_itivp_dot/tree/kr21/ITIVP_collection.postman_collection.json

ВЫВОДЫ

В ходе выполнения индивидуальной практической работы были успешно изучены и применены на практике принципы обеспечения безопасности современных веб-приложений. Разработано и настроено защищенное *REST API* на базе *Node.js*, *Express* и реляционной СУБД *PostgreSQL* (*Neon.tech*) с использованием *ORM Sequelize*.

Была реализована надежная, независимая от состояния (*stateless*) система аутентификации пользователей с использованием стандарта *JWT* (*JSON Web Token*). Разработана система промежуточных обработчиков (*middleware*) для реализации ролевого контроля доступа (*RBAC — user*, *moderator*, *admin*), обеспечивающая надежную защиту административных и преподавательских маршрутов от несанкционированного использования.

Также были успешно внедрены механизмы защиты от распространенных векторов атак: настроена строгая валидация входящих данных через *Joi* для предотвращения обработки некорректных запросов, реализована комплексная защита от атак типа *Brute-Force* путем временной блокировки учетной записи после серии неудачных попыток входа с возвратом статус-кода 429 (*Too Many Requests*), а также подключены средства сетевой защиты (*Helmet*, *Express-Rate-Limit*, *CORS*). Все реализованные механизмы безопасности были успешно протестированы в *Postman* и через *Fetch API*, что подтверждается корректными *HTTP*-ответами при различных сценариях использования *API*.

.